Conference Paper Title

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

***Abstract*—Deploying AI models as microservices at the network edge faces an inherent conflict: the demand for high-quality, real-time inference versus the reality of strictly constrained, highly volatile node resources and bursty traffic. To address these dynamics, existing deployment strategies either rely on static model pools that lack agility or utilize Neural Architecture Search (NAS) to customize hardware-aware models. However, conventional NAS is computationally prohibitive for real-time, on-device model evaluation and dynamic switching. To bridge this gap, this paper proposes a novel cloud-edge collaborative framework that empowers edge nodes with millisecond-level, on-demand model switching capabilities. By decoupling heavy architecture training from a cloud-hosted supernet, our method leverages an enhanced Zero-Cost Proxy (ZCP) method for ultra-fast, lightweight candidate evaluation directly on edge nodes. Furthermore, we design a state-driven dynamic heuristic scheduler that strictly enforces transient memory and compute limits while adaptively balancing inference accuracy and service latency via traffic-aware penalty reweighting. Extensive experiments demonstrate that our framework significantly outperforms representative baselines in end-to-end latency, resource utilization, and robustness under varied traffic perturbations and dynamic resource conditions, validating real-time state-driven orchestration as a highly effective paradigm for resilient Edge AI.**

***Index Terms*—component, formatting, style, styling, insert.**

I. INTRODUCTION

With the rapid advancement of the Internet of Things (IoT) and 5G/6G communication technologies, the volume of data generated at the network edge has experienced explosive growth. To meet the demands of time-sensitive applications, such as autonomous driving and smart manufacturing, Edge AI [1], [2] has emerged. It focuses on deploying deep learning models as lightweight microservices on edge nodes in close proximity to data sources. However, practical edge computing environments face significant dynamic challenges. On the one hand, the physical resources of heterogeneous edge nodes (e.g., computational capacity and memory) are strictly constrained. On the other hand, a massive number of concurrent AI requests with stringent Service Level Agreements (SLAs) frequently emerge within the network. Such highly dynamic and time-varying traffic fluctuations necessitate that edge networks possess the capabilities for on-demand model orchestration and dynamic microservice scheduling [3], [4].

To provide high-performance AI services adapted to the physical constraints of edge nodes, existing cloud-edge orchestration schemes typically focus on jointly optimizing microservice deployment and request routing. For instance, some works emphasize energy-delay trade-offs in resource-constrained edge environments [5], [6], while others explore service mesh orchestration for large-scale and symmetric cloud systems [7], [8]. Furthermore, collaborative routing strategies have been proposed to handle dynamic call graphs and fluctuating traffic [9]. However, these approaches often assume static AI models and frequently treat AI services simply as fixed microservices [10], neglecting their architectural adaptability under varying resources. While Neural Architecture Search (NAS) [11] and supernet paradigms [12], [13] offer massive candidate subnets to overcome this, they suffer from severe response lag in online environments due to computationally expensive gradient-based evaluations. To circumvent this overhead, current frameworks often degrade to static matching based on offline finite model pools. However, these coarse-grained pools struggle to handle time-varying bursty traffic, failing to balance queueing delay and inference accuracy. Although Zero-Cost Proxies (ZCP) [14], [15] enable millisecond-level accuracy quantification, they remain confined to the pure architecture search domain. Crucially, they lack effective coupling with system-level dynamic microservice reloading, hard physical constraints, and queueing dynamics.

To address the conflict between the prohibitively high search overhead and the requirement for dynamic edge deployment, this paper proposes a cloud-edge collaborative deployment framework based on ZCP and dynamic scheduling. Specifically, this framework pre-trains a SuperNet offline in the cloud, and upon encountering traffic fluctuations, the edge triggers on-demand sampling and rapid evaluation mechanisms. Leveraging ZCP, the system quantifies the accuracy potential and physical overhead of candidate subnets at the millisecond level. Building on this, the edge scheduler applies hard-constraint filtering based on transient available memory and computational upper bounds, and uses a dynamic heuristic algorithm to detect traffic variations, thereby adaptively adjusting the weights of soft constraints. Ultimately, the edge selects the optimal subnet and fetches its corresponding weights from the cloud on demand to complete rapid hot-reload deployment.

The main contributions of this paper are summarized as follows:

- We enhance a fast ZCP method to meet the demands of edge scenarios, effectively decoupling the architecture search from the online deployment process. This enables millisecond-level performance estimations on edge nodes,

Identify applicable funding agency here. If none, delete this.

saving critical time for model transmission and reloading.

- We propose a hard-constraint-driven dynamic heuristic scheduling algorithm. It filters subnets via real-time memory and compute limits, and adaptively tunes soft constraints based on traffic. This ensures proper degradation and sustains overall throughput during traffic bursts.
- We construct a cloud-edge decoupled lightweight deployment process. Edge nodes run only a minimal ZCP and parser to fetch subnets on demand. This eliminates local supernet storage, significantly reducing resource burdens while maintaining high service availability.

II. SYSTEM MODEL AND PROBLEM FORMULATION

This section proposes a cloud-edge collaborative on-demand microservice orchestration framework for delay-sensitive applications. This framework offloads the computation-intensive SuperNet pre-training to the cloud, while delegating the lightweight on-demand sampling, evaluation, and dynamic scheduling to the edge.

A. System Model

Suppose the system consists of a cloud center and a set of heterogeneous edge nodes denoted by $\mathcal{N} = \{n_1, n_2, \ldots, n_N\}$. The cloud offline pre-trains a supernet $\mathcal{W}$ containing a massive number of candidate subnets for the edge to fetch on demand. At any scheduling slot t, the transient physical available resource constraints of edge node $n_i \in \mathcal{N}$ are defined by a tuple: $Node_i(t) = \langle M_i^{max}(t), C_i^{max} \rangle$, which represent the currently available memory and the peak computational capacity (measured in FLOPS), respectively. Specifically, let M_{total} denote the total memory capacity of node n_i, and $M_{used}(t)$ denote the memory currently occupied by background processes and other multi-tenant services. The transient available memory is thus strictly bounded by $M_i^{max}(t) = M_{total} - M_{used}(t)$. When the system senses traffic fluctuations and triggers scheduling, the edge performs heuristic sampling on the supernet $\mathcal{W}$ and rapidly abstracts the candidate subnet a_k into a triple feature representation via the ZCP and parser:

$$a_k = \langle S_k^{proxy}, P_k^{model}, F_k^{flops} \rangle \tag{1}$$

where S_k^{proxy} is the theoretical accuracy potential score evaluated by ZCP; P_k^{model} and F_k^{flops} denote the number of parameters (memory requirement) and the number of floating-point operations (computational demand) of the subnet, respectively. To guarantee the time sensitivity of microservices under highly dynamic time-varying traffic $\lambda_i(t)$, the system sets an end-to-end delay SLA (denoted as T_{SLA}). For subnet a_k, its service rate on node n_i is approximated as $\mu_{i,k} = \frac{C_i^{max}}{F_k^{flops}}$. While neural network inference time on isolated hardware is typically deterministic, practical edge nodes often exhibit service time variability due to OS-level context switching, resource contention, and concurrent multi-tenant workloads. To capture this stochasticity while maintaining analytical tractability, we adopt the M/M/1 queueing model, which provides a conservative approximation for the system's average response time compared to deterministic execution. Thus, the average response time at slot t must satisfy:

$$T_{req} = \frac{1}{\mu_{i,k} - \lambda_i(t)} = \frac{1}{\frac{C_i^{max}}{F_k^{flops}} - \lambda_i(t)} \leq T_{SLA} \tag{2}$$

Consequently, the upper bound of the maximum tolerable computation $F_i^{max}(t)$ that the node can allocate to the target subnet is dynamically compressed by the traffic, which is derived as:

$$F_i^{max}(t) = \frac{C_i^{max}}{\lambda_i(t) + \frac{1}{T_{SLA}}} \tag{3}$$

Equation (3) establishes the computational limit from a physical boundary perspective: when the concurrent traffic $\lambda_i(t)$ surges, the upper bound of available computation shrinks sharply, and the system is forced to filter out heavy networks and select lightweight subnets with smaller F_k^{flops}, thereby mitigating the exacerbation of queueing congestion.

B. Problem Formulation

Let $x_{i,k}(t) \in \{0, 1\}$ be a boolean decision variable indicating whether subnet a_k is selected and fetched for node n_i at slot t (the total size of the candidate pool is denoted as K). The global scheduling problem is modeled as a constrained online joint optimization problem:

$$\max_{\mathbf{x}} \quad \sum_{t=1}^{T} \sum_{i=1}^{N} \sum_{k=1}^{K} x_{i,k}(t) \cdot \widehat{S}_k^{proxy} \tag{4}$$

$$\text{s.t.} \quad \sum_{k=1}^{K} x_{i,k}(t) = 1, \quad \forall i \in \mathcal{N}, \forall t \tag{5}$$

$$\sum_{k=1}^{K} x_{i,k}(t) \cdot P_k^{model} \leq M_i^{max}(t), \quad \forall i \in \mathcal{N}, \forall t \tag{6}$$

$$\sum_{k=1}^{K} x_{i,k}(t) \cdot F_k^{flops} \leq F_i^{max}(t), \quad \forall i \in \mathcal{N}, \forall t \tag{7}$$

Here, the objective function (4) aims to maximize the accuracy potential of the selected models. Constraint (5) guarantees that only one optimal architecture is activated per node at a time. Constraints (6) and (7) constitute the hard-constraint filtering pool, strictly ensuring that the overhead of the selected subnet does not exceed the node's transient available memory and the queueing computational upper bound defined by Equation (3).

III. ONLINE ZERO-COST ARCHITECTURE EVALUATION

To enable the cloud-edge collaborative pipeline of "edge search, cloud delivery, and edge deployment" to meet strict real-time requirements, the system must perform ultra-fast multi-dimensional attribute evaluation of candidate architectures at the edge, thereby generating the attribute tuple $a_k = \langle S_k^{proxy}, P_k^{model}, F_k^{flops} \rangle$ that encapsulates accuracy potential and resource overhead. Since the computation volume (F_k^{flops}) and parameter size (P_k^{model}) of the candidate subnet a_k are constant values once its operator-level topology is determined,

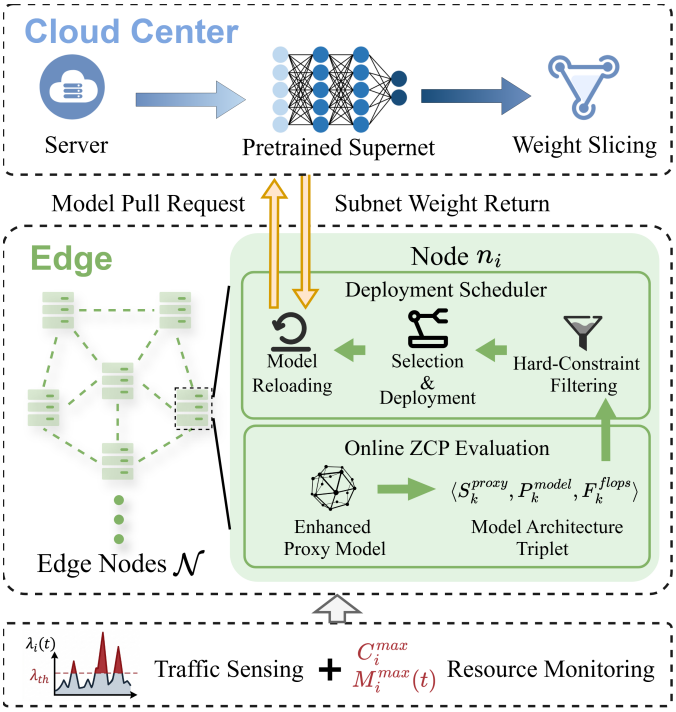


Fig. 1. System architecture of the proposed cloud-edge collaborative deployment framework based on ZCP and dynamic scheduling. The cloud center pre-trains the supernet offline, while the edge node relies on traffic sensing and resource monitoring to perform online ZCP evaluation and heuristic scheduling, ultimately reloading the optimal subnet on demand.

the edge scheduler only needs to conduct a lightweight static topology traversal with $\mathcal{O}(L)$ complexity to deterministically obtain these two physical overhead features. Therefore, this section primarily focuses on how to circumvent expensive online gradient training and achieve accurate model accuracy potential evaluation (S_k^{proxy}) tailored to heterogeneous edge scenarios under ultra-low latency.

Traditional ZCP methods often adopt static, unified metrics, making it difficult to adapt to the complex and variable hardware constraints and traffic fluctuations in edge computing environments. To this end, this framework introduces the GSTC proxy model [15] and proposes a dynamic adaptation mechanism tailored for heterogeneous edge states.

After the edge scheduler generates the Directed Acyclic Graph (DAG) for the candidate subnet a_k, the evaluator utilizes the locally cached operator information retention rate (η) to efficiently extract four fundamental physical quantities characterizing the information flow of the network topology: the total architectural path information G, the path information difference S, the total information received by the output node T, and the information difference received by the output node C. To ensure that the proxy evaluation naturally aligns with the transient resource constraints and service demands of the current edge node, the system dynamically allocates weights $(\alpha, \beta, \gamma, \delta)$ to each component, or even performs dimensional pruning, to calculate a customized proxy score:

$$S_k^{proxy} = \alpha \cdot G + \beta \cdot S + \gamma \cdot \widehat{T} + \delta \cdot \widehat{C} \tag{8}$$

where $\widehat{T}$ and $\widehat{C}$ are min-max normalized terms based on the upper and lower bounds of the global search space. Thanks to the high scalability of this evaluation mechanism, the scheduler can dynamically map to the most suitable evaluation scheme based on the node's real-time multi-dimensional states. The following lists three of the most representative typical scenarios:

Low-Latency & Resource-Constrained Scheme: When edge nodes face extreme physical resource constraints or are experiencing peak concurrent traffic, complex network topologies, despite enhancing information difference, can lead to higher memory requirements. In such cases, the system triggers a dimensionality reduction and pruning strategy, reducing the weights of the structural difference terms and potentially introducing a negative decay penalty to the total information G. This scheme guides the scheduler from the evaluation source to favor candidate architectures with shallower layers and simpler topologies, thereby maximizing the system throughput and overload resistance.

High-Accuracy & Relaxed-Latency Scheme: For nodes with relatively abundant computational resources and stringent requirements for inference accuracy, the primary optimization objective shifts to enhancing the representation capability of the model. The physical quantities G and $\widehat{T}$ are highly positively correlated with the network depth and feature extraction capacity. Therefore, the system assigns higher weight factors to the total information and the output information. This scheme can capture high-complexity subnets within the search space that exhibit lossless information flow and extremely deep hierarchical structures, ensuring the maximum utilization of the underlying hardware computational capacity within resource constraints to trade off ultimate accuracy.

Balanced Robust Scheme: When node resources and traffic are in a steady state, the system adopts the full set of evaluation components with balanced weights. This scheme jointly considers network depth (characterized by G and T) and the diversity of residual structures (characterized by S and C), achieving an optimal comprehensive trade-off between theoretical accuracy potential and structural redundancy to provide a highly robust normalized candidate set.

Through the aforementioned dynamic adaptation mechanism, the system decomposes the computationally heavy online joint optimization problem. Edge nodes can generate a multi-dimensional set of candidate architectures within milliseconds that not only comply with physical constraints but also possess topological characteristics naturally suited to transient traffic scenarios. This process secures a substantial time margin for the subsequent on-demand fetching of models from the cloud.

IV. CLOUD-EDGE COLLABORATIVE HEURISTIC DEPLOYMENT

Upon triggering a scheduling event, the edge scheduler first evaluates the transient resource state (e.g., $M_i^{max}(t)$) and concurrent traffic $\lambda_i(t)$ to dynamically configure the

GSTC proxy weights $(\alpha, \beta, \gamma, \delta)$ according to the context-aware schemes outlined in Section III. By executing this lightweight static topology traversal, the system generates the customized accuracy potential score S_k^{proxy} for each candidate on heterogeneous nodes. Based on these extracted context-aware attributes, the edge scheduler then executes a dynamic deployment pipeline comprising three stages: hard-constraint filtering, heuristic weighted selection, and on-demand fetching. This dynamic mechanism copes with bursty traffic micro-states and realizes fine-grained adaptive proper degradation, the specific process is as shown in Algorithm 1.

Algorithm 1 Cloud-Edge Collaborative Dynamic Heuristic Scheduling

Require: Supernet candidate set $\mathcal{A}$, transient state $M_i^{max}(t), \lambda_i(t)$, SLA bound T_{SLA}, safe threshold λ_{th}

Ensure: Optimal subnet encoding a^* for current time slot t.

Determine GSTC weights $(\alpha, \beta, \gamma, \delta)$ based on transient resource limits and traffic;
for each candidate $a_k \in \mathcal{A}$ **do**
  Calculate customized proxy score S_k^{proxy} via Eq. (8);
  Extract physical attributes to form tuple $a_k = \langle S_k^{proxy}, P_k^{model}, F_k^{flops} \rangle$;
end for
Calculate dynamic compute limit $F_i^{max}(t)$ via Eq. (3);
Initialize feasible subset $\mathcal{A}_i^*(t) \leftarrow \varnothing$;
for each candidate $a_k \in \mathcal{A}$ **do**
  if $P_k^{model} \leq M_i^{max}(t)$ AND $F_k^{flops} \leq F_i^{max}(t)$ **then**
    $\mathcal{A}_i^*(t) \leftarrow \mathcal{A}_i^*(t) \cup \{a_k\}$;
  end if
end for
if $\mathcal{A}_i^*(t)$ is empty **then**
  return System Failure / OOM
end if
Calculate penalty weights $w_2(t)$, $w_3(t)$, and normalized weight $w_1(t)$ via Eq. (10);
for each candidate $a_k \in \mathcal{A}_i^*(t)$ **do**
  Normalize attributes to $\hat{S}_k^{proxy}, \hat{F}_k^{flops}, \hat{P}_k^{model} \in [0, 1]$;
  Calculate utility score $U(a_k)$ via Eq. (9);
end for
Select optimal architecture: $a^* = \arg\max_{a_k} U(a_k)$;
Send a^* encoding to the cloud server;
Cloud slices weights from $\mathcal{W}$ and edge hot-reloads a^*;
return a^*

First, in the physical constraint filtering stage, the engine processes the candidate subnet set $\mathcal{A}$ in $\mathcal{O}(|\mathcal{A}|)$ time based on transient resource bounds. It eliminates subnets exceeding the memory limit ($P_k^{model} > M_i^{max}(t)$) to prevent Out-of-Memory (OOM) failures, and those violating the queueing computation limit ($F_k^{flops} > F_i^{max}(t)$) to strictly guarantee SLA delay boundaries under the transient arrival rate $\lambda_i(t)$. This safely prunes the global search space into a feasible real-time subset $\mathcal{A}_i^*(t)$.

Subsequently, the process enters the dynamic heuristic degradation stage. To optimally trade off accuracy potential, inference latency, and memory footprint within $\mathcal{A}_i^*(t)$, we design a utility function based on Global Boundary Normalization:

$$U(a_k) = w_1(t) \cdot \widehat{S}_k^{proxy} - w_2(t) \cdot \widehat{F}_k^{flops} - w_3(t) \cdot \widehat{P}_k^{model} \tag{9}$$

where $\widehat{(\cdot)}$ denotes min-max normalization over the entire cloud supernet to prevent local scale distortion, and $\widehat{S}_k^{proxy}$ is the theoretical accuracy score evaluated by our customized GSTC. To handle microscopic fluctuations, the penalty weights are dynamically driven by transient traffic $\lambda_i(t)$ and resource utilization:

$$\begin{aligned} w_2(t) &= c_1 \cdot \exp\left(\max\left(0, \frac{\lambda_i(t) - \lambda_{th}}{\lambda_{th}}\right)\right), \\ w_3(t) &= c_2 \cdot \frac{M_{used}(t)}{M_{total}} \end{aligned} \tag{10}$$

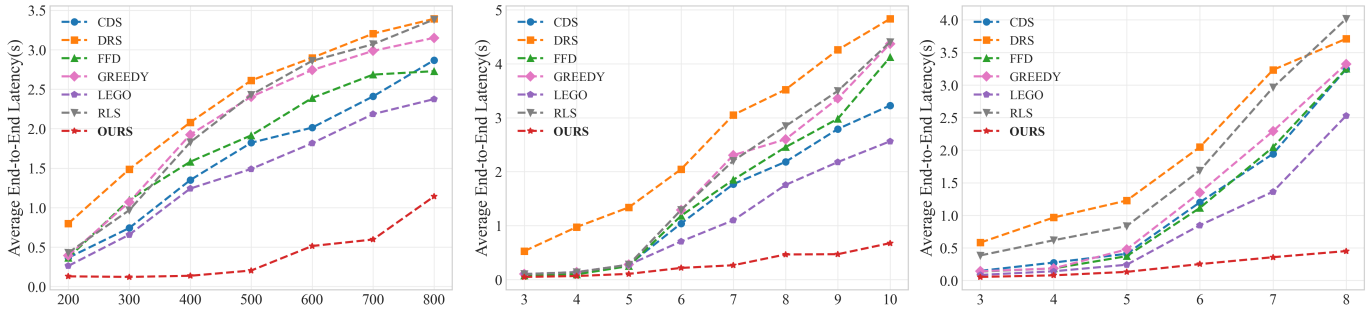
with the normalized accuracy weight $w_1(t) = \frac{1}{1+w_2(t)+w_3(t)}$, where λ_{th} is a pre-calibrated safe traffic threshold. This normalized weight distribution intrinsically forms a self-adaptive penalty mechanism. When traffic is stable ($\lambda_i(t) \leq \lambda_{th}$), $w_2(t)$ remains constant, prioritizing the GSTC-evaluated accuracy. Conversely, during traffic bursts ($\lambda_i(t) > \lambda_{th}$), $w_2(t)$ grows exponentially, severely penalizing computation and suppressing $w_1(t)$. This proactively triggers system-level degradation, trading accuracy for lightweight architectures to sustain high throughput and service availability.

Finally, in the on-demand fetching stage, the scheduler identifies the optimal subnet $a^* = \arg\max_{a_k \in \mathcal{A}_i^*(t)} U(a_k)$ and sends its Architecture Encoding to the cloud. The cloud server instantly slices the corresponding weights from the pre-trained supernet $\mathcal{W}$ and transmits them to the edge. By bypassing the massive gradient training overhead via the GSTC proxy, the entire "search-decision-fetch" duration is strictly compressed, enabling agile hot-reload microservice deployment within a tolerable time window.

V. EXPERIMENTAL ANALYSIS

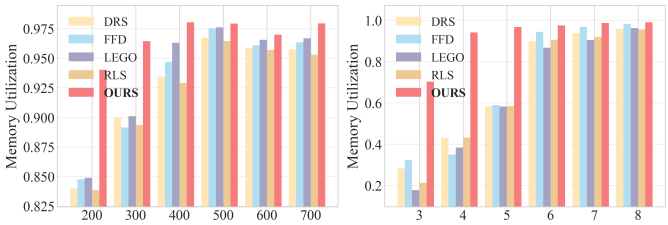
A. Experimental Settings

We pre-train a supernet offline and build a multi-task model library for online deployment. To emulate realistic microservice traces, we preprocess call graphs into independent pipelined request chains. The number of service types ranges from 3 to 15, each request chain has length mostly in 2–9, and the number of requests per run ranges from 10 to 1000. For traffic perturbation, the arrival rate varies from 100 to 800 requests/s, and per-chain rates are sampled from a Dirichlet distribution ($\alpha = 1$). We compare six baselines: First-Fit Decreasing (**FFD**), Chain-aware Deployment Scoring (**CDS**), Random Local Search (**RLS**), **Greedy**, Dynamic Resource Scaling (**DRS**), and **LEGO** [16]. They use fixed models, while our method performs dynamic model selection under hard/soft constraints.



(a) Arrival Rate of Requests (b) Number of Request Types (c) Length of Requests

Fig. 2. Average end-to-end latency under three perturbation axes: (a) arrival rate, (b) number of request types, and (c) request chain length.



(a) Arrival Rate of Requests (b) Number of Request Types

Fig. 3. Average memory utilization under two perturbation axes: (a) arrival rate and (b) number of request types.

B. Static Performance

We first report static-condition results from two aspects: end-to-end latency and resource utilization.

End-to-End Latency: As traffic increases, the hard bound $F_i^{max}(t)$ in Eq. (3) narrows the feasible compute region, and the scheduler naturally shifts to lighter subnets. Therefore, latency remains consistently below all baselines across the three perturbation axes in Fig. 2. Under arrival-rate perturbation (Fig. 2(a)), our latency grows much more slowly as load increases, while all baselines rise rapidly and remain at high levels. For example, at $\lambda = 400$, our delay is 0.14 s, about 93% lower than the baseline mean (2.02 s). Similar gaps are observed on the other two axes, where baseline methods also stay markedly higher. These gains come from adaptive subnet selection (Eq. (8)) and dynamic penalty reweighting (Eq. (10)), which prioritize queue stability under heavy traffic.

Resource Utilization: Under the same static settings, Fig. 3 reflects a clear adaptive-deployment behavior. Under light load, our scheduler tends to select higher-accuracy subnets, so memory utilization is intentionally higher and edge resources are used more fully. For example, at $\lambda = 200$, our utilization reaches 94%. By contrast, fixed-model baselines often under-utilize memory at low load and then move close to saturation as load increases. This trend is consistent with Eq. (9), where low-load settings favor quality-oriented model choices and high-load settings increase resource penalties.

C. Scalability

We analyze scalability along two complementary dimensions: horizontal scaling and vertical scaling.

Horizontal Scaling: We first evaluate performance changes when expanding the number of edge nodes under fixed load. As shown in Fig. 4, three topologies are considered: Small ($N = 3$), Medium ($N = 5$), and Large ($N = 7$). As the scale increases, the network provides more aggregate resources, and the service quality of all methods improves accordingly. Even in this case, our method remains consistently better than all baselines, because it can dynamically switch to higher-accuracy models when resources are available. In terms of median fitness, our method achieves 2.41, 3.33, and 3.94 from Small to Large, while the best baseline reaches 1.74, 2.09, and 2.84, corresponding to relative gains of 38%, 59%, and 39%, respectively.

Vertical Scaling: We further quantify resource impact by using a unified resource factor to adjust per-node resource scale in Fig. 5. As the factor increases, available resources grow. Our method remains lower throughout, since it can dynamically deploy models that better match current resource conditions. At factor 0.5, our delay is about 1.46 s, while baselines are around 3.2 s, corresponding to an improvement of roughly 54%. At factor 1.5, delays of all methods become close, indicating that resources are already abundant and the performance gap naturally narrows. This result is consistent with the horizontal-scaling experiment and confirms that adaptive model switching improves robustness under both topology and per-node resource variations.

D. Dynamic Adaptation

We further evaluate service capability under dynamic load conditions, as shown in Fig. 6. When the system enters stressed periods, delays of baseline methods increase sharply, while our curve stays consistently lower. At a representative high-load state, our delay is 1.28 s, whereas all baselines remain above 1.8 s. This indicates that our method can adjust deployment strategy according to dynamic environments, achieving stronger adaptability and better overall performance.

VI. CONCLUSION

This paper proposed a cloud-edge collaborative framework integrating ZCP evaluation and dynamic heuristic scheduling to address the deployment of edge AI microservices under bursty traffic and strict resource constraints. By enhancing the ZCP method for heterogeneous edge states, our

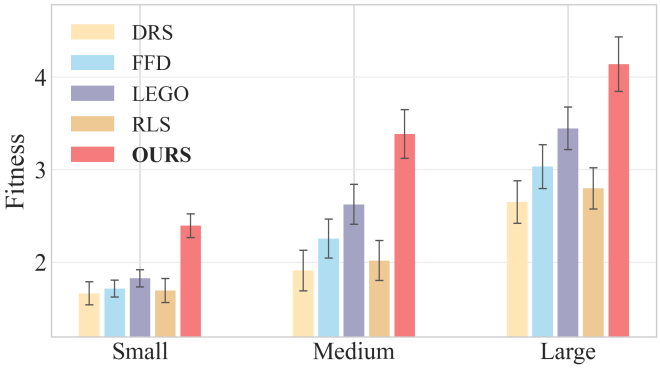


Fig. 4. Fitness distribution under three network scales (Small: $N = 3$, Medium: $N = 5$, Large: $N = 7$). Each box represents 100 independent trials.

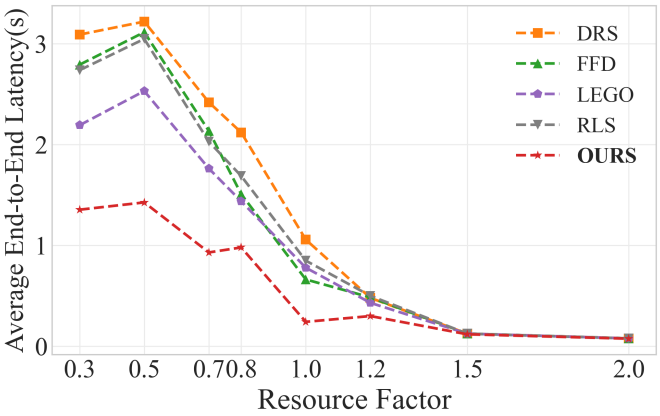


Fig. 5. Average end-to-end latency under varying resource constraints.

approach enables millisecond-level performance analysis of candidate models for nodes. Furthermore, a hard-constraint-driven scheduler adaptively reweights soft penalties based on transient traffic, guaranteeing system feasibility while optimally trading off inference accuracy and latency. Finally, an on-demand model reloading strategy avoids local supernet storage overhead. Extensive experiments demonstrate that our approach significantly outperforms representative baselines in end-to-end latency, resource utilization, and robustness across varied traffic perturbations, node scales, and dynamic resource conditions. These findings validate that real-time state-driven dynamic deployment is a highly effective paradigm for scalable Edge AI orchestration.

ACKNOWLEDGMENT

The preferred spelling of the word "acknowledgment" in America is without an "e" after the "g". Avoid the stilted expression "one of us (R. B. G.) thanks ...". Instead, try "R. B. G. thanks...". Put sponsor acknowledgments in the unnumbered footnote on the first page.

REFERENCES

[1] E. Li, L. Zeng, Z. Zhou, and X. Chen, "Edge ai: On-demand accelerating deep neural network inference via edge computing," *IEEE transactions on wireless communications*, vol. 19, no. 1, pp. 447–457, 2019.

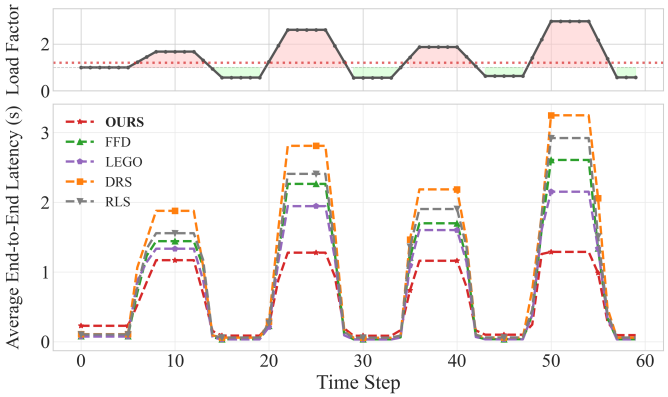


Fig. 6. Dynamic adaptation performance over time-varying traffic and resource conditions.

[2] V. Gujar, "Smarter at the edge: Evaluating decentralized ai deployment models in federated, hierarchical, microservices and serverless edge ai architectures," *Scientific Computing and Instrumentation*, vol. 14, no. 6, pp. 1–16, June 2025.

[3] Y. Hu, L. Hou, J. Hu, M. Ren, M. Hu, C. Cai, and K. Peng, "Time-varying microservice orchestration with routing for dynamic call graphs via multi-scale deep reinforcement learning," *IEEE Transactions on Services Computing*, vol. 18, no. 5, pp. 3276–3291, 2025.

[4] K. Peng, J. Hu, Z. Wang, Y. Hu, M. Hu, Z. Xiong, and Z. Chen, "Collaborative orchestration of microservices and ai services in edges: A dual-time-scale reinforcement learning approach," *IEEE Transactions on Services Computing*, pp. 1–14, 2026.

[5] L. Wang, X. Liu, H. Ding, Y. Hu, K. Peng, and M. Hu, "Energy-delay-aware joint microservice deployment and request routing with dvfs in edge: A reinforcement learning approach," *IEEE Transactions on Computers*, vol. 74, no. 5, pp. 1589–1604, 2025.

[6] Y. Hu, J. Hu, Y. Yang, M. Hu, K. Peng, T. Zheng, C. Cai, and Z. Xiong, "Energy-aware service mesh deployment and online request routing in edge: a hierarchical deep reinforcement learning approach," *IEEE Transactions on Mobile Computing*, pp. 1–18, 2026.

[7] K. Peng, Y. Hu, H. Ding, H. Chen, L. Wang, C. Cai, and M. Hu, "Large-scale service mesh orchestration with probabilistic routing in cloud data centers," *IEEE Transactions on Services Computing*, vol. 18, no. 2, pp. 868–882, 2025.

[8] K. Peng, T. Liao, M. Ren, Y. Hu, L. Wu, M. Hu, and H. Jiang, "Symmetric orchestration under service mesh paradigm: Empowering massive online applications in edge clouds," *IEEE Transactions on Mobile Computing*, vol. 25, no. 3, pp. 4300–4316, 2026.

[9] Y. Hu, H. Ding, H. Chen, J. He, M. Hu, C. Cai, and K. Peng, "Collaborative orchestration with probabilistic routing for dynamic service mesh in clouds," in *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*, 2025, pp. 1–10.

[10] K. Peng, X. Liu, D. Han, Y. Hu, M. Hu, C. Cai, and Z. Xiong, "Hybrid orchestration of ai services and microservices in cloud-edge collaboration," *IEEE Transactions on Mobile Computing*, pp. 1–17, 2026.

[11] P. Ren, Y. Xiao, X. Chang, P.-Y. Huang, Z. Li, X. Chen, and X. Wang, "A comprehensive survey of neural architecture search: Challenges and solutions," *ACM Computing Surveys (CSUR)*, vol. 54, no. 4, pp. 1–34, 2021.

[12] H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han, "Once-for-all: Train one network and specialize it for efficient deployment," in *International Conference on Learning Representations*, 2020.

[13] J. Yu, L. Yang, N. Xu, J. Yang, and T. Huang, "Slimmable neural networks," in *International Conference on Learning Representations*, 2019.

[14] M. S. Abdelfattah, A. Mehrotra, Ł. Dudziak, and N. D. Lane, "Zero-cost proxies for lightweight nas," in *International Conference on Learning Representations*, 2021.

[15] Y. Yang, J. Lu, Z. Jing, L. Wang, D. Han, M. Hu, and K. Peng, "A zero-cost proxy model for nas based on information quantity quantization encoding," *Expert Systems with Applications*, vol. 307, p. 130989, 2026.

[16] Y. Yu, J. Yang, C. Guo, H. Zheng, and J. He, "Joint optimization of service request routing and instance placement in the microservice

system," *Journal of Network and Computer Applications*, vol. 147, p. 102441, 2019.